QUALITY ASSURANCE TESTING DOCUMENTATION (QATD)

UMHackathon 2026

DaddiesTrip

AI-Powered Cross-Border Travel Orchestration & Group Expense Platform

Team: SugarDaddies

Table of Content

Document Control	3
PRELIMINARY ROUND (Test Strategy & Planning)	3
1. Scope & Requirements Traceability	3
2. Risk Assessment & Mitigation Strategy	3
3. Test Environment & Execution Strategy	5
4. CI/CD Release Thresholds & Automation Gates	5
5. Test Case Specifications (Drafts)	6
6. AI Output & Boundary Testing (Drafts)	7
7. Test Strategy & Plan Sign-Off	8

Document Control

System Under Test (SUT)	UMHackathon 2026 - Team SugarDaddies
Team Repo URL	https://github.com/SkyLee310/UMHackathon-DaddiesTrip
Live Deployment URL	https://daddies-trip.vercel.app/

Objective: The primary objective is to ensure that DaddiesTrip can reliably handle conversational trip planning, AI agent pipeline execution, multi-currency expense splitting, and simulated payment settlement across concurrent users, with CI/CD checkpoints enforcing quality at every merge and deployment.

1.1 In-Scope Core Features

- **Conversational Trip Planning:** Natural language prompt input, AI field extraction (destination, dates, group size, budget), day-by-day itinerary generation via the 4-agent pipeline.
- **AI Agent Pipeline:** Analyzer Agent validation, Planner Agent itinerary generation, Booking Agent flight/hotel enrichment, Edge Agent hallucination detection and correction.
- **Real-Time SSE Streaming:** Progressive server-sent events (progress, partial_itinerary, partial_flights, complete, error) delivered to the frontend as each agent completes.
- **Multi-Currency Expense Splitting:** Per-person cost calculation with live exchange rates and offline fallback for 17+ currencies.
- **Simulated Payment Settlement:** Card validation, payment flow, booking confirmation animation via /api/settle endpoint.

1.2 Out-of-Scope

- In-app real-time chat between group members
- User account management: profile editing, saved trip history, delivery address management
- Integration with live booking APIs (Amadeus, Booking.com) for real seat/room availability
- Promotional offers, discount codes, and voucher systems
- Native iOS and Android mobile application testing

2. Risk Assessment & Mitigation Strategy

Technical Risk	Likelihood (1-5)	Severity (1-5)	Risk Score (L×S)	Mitigation Strategy	Testing Approach
Ilmu AI API rate-limited or unavailable — core pipeline fails for all users.	4	5	**20 (Critical)**	Graceful degradation: emit error SSE event with user-facing retry UI. Monitor API status page. Add request retry with exponential backoff in base_agent.py.	Simulate API timeout in unit test; verify error SSE event is emitted and frontend displays retry state correctly.
LLM hallucination — Booking Agent returns identical costs (RM25 repetition bug) or invalid flight routes.	4	4	**16 (Critical)**	Edge Agent heuristic checks detect and auto-correct RM25 repetition, same departure/return airport, and missing day fields before the complete event is emitted.	Inject known hallucination patterns into Booking Agent output; verify Edge Agent corrects them. Validate final trip object schema.

Technical Risk	Likelihood (1-5)	Severity (1-5)	Risk Score (L×S)	Mitigation Strategy	Testing Approach
SSE connection drops mid-pipeline — user receives incomplete trip data.	3	4	**12 (High)**	Frontend displays partial results (partial_itinerary, partial_flights) already received. Error event triggers a user-friendly message with retry option.	Simulate connection drop after partial_itinerary event; verify frontend preserves received data and shows retry prompt.
Oversized user prompt crashes the pipeline (buffer overflow).	3	4	**12 (High)**	Orchestrator truncates prompts exceeding 1,500 words before any LLM call. Input length is validated before pipeline execution.	AI-01: Submit a 9,000-word prompt; verify truncation to 1,500 words and successful pipeline completion.
Invalid card number accepted by /api/settle — false payment confirmation.	2	5	**10 (Medium)**	Ledger Service applies deterministic card validation (Luhn algorithm check) before settlement. Invalid cards return a 400 error with descriptive message.	TC-02: Submit a sub-16-digit card; verify 400 response and descriptive error message displayed in UI.
Currency API unavailable — expense splitting uses stale or zero rates.	3	3	**9 (Medium)**	Offline fallback table covers 17+ currencies. If live rates fail, fallback rates are used transparently and a UI note indicates rates may not be current.	Mock currency API to return HTTP 503; verify fallback rates are used and UI displays appropriate notice.
Pipeline latency > 90 seconds — SSE connection times out on client.	3	3	**9 (Medium)**	Railway persistent containers have no hard timeout. Frontend shows per-agent progress states to manage user expectation during long runs.	TC-03 (NFR): Measure p95 pipeline latency under 50 concurrent requests; verify Railway container handles load without timeout.
API keys exposed in frontend build or repository.	2	4	**8 (Medium)**	All secrets stored as Railway/Vercel environment variables; .env.example committed, .env in .gitignore. No secrets in Vite build output.	Security gate: run secret scanning on repository; verify no API keys in built frontend bundle.

Risk Assessment Scoring Criteria

Score	Likelihood	Score	Severity
1	Rare	1	Impact is Negligible
2	Unlikely	2	Impact is Minor
3	Possible	3	Moderate Impact
4	Likely	4	Major Impact
5	Almost Certain	5	Critical Failure of System

Risk Score = Likelihood × Severity

Risk Level Reference

Risk Score	Risk Level	Recommended Action
1 – 5	**Low**	Monitor only. Acceptable risks.
6 – 10	**Medium**	Mitigate + Testing
11 – 15	**High**	Must mitigate; thorough testing required
16 – 25	**Critical**	Highest priority. Extensive testing required.

3. Test Environment & Execution Strategy

- **Unit Test**
  - Scope: Individual agent logic (Analyzer field extraction, Edge Agent hallucination detection, Ledger Service card validation, prompt truncation in Orchestrator).
  - Execution: Tests written in PyTest. Executed locally during development and automatically in the CI pipeline on every push to the repository.
  - Isolation: External dependencies (Ilmu AI API, currency API) are mocked using unittest.mock. FastAPI endpoints tested with TestClient.
  - Pass Condition: Happy cases, negative cases with correct error responses (e.g., '400 Invalid Card Number'), and edge cases (prompt > 1,500 words, missing required fields) all pass with expected return formats.

- **Integration Test**
  - Scope: End-to-end SSE pipeline — from POST /api/plan-trip-stream through all 4 agents to the complete event emission.

  - Execution: Performed after unit tests pass and agent branches are merged. Uses real FastAPI TestClient with SSE event parsing.
  - Workflow: Real agent calls are made (no mocking) to verify inter-agent data handoff — Analyzer output becomes Planner input, Planner output becomes Booking input, etc.
  - Pass Condition: complete SSE event received with a valid trip object containing all required fields (days, flight_options, budget_summary, per_person_cost).

- **Test Environment (CI/CD Practice):**
  - Local Testing: Manual testing on localhost (backend: uvicorn at port 8000, frontend: Vite at port 5173) with a proxied API connection.
  - Staging/CI: GitHub Actions triggers the PyTest suite on every push to any branch.
  - CI/CD Automated Pipeline: GitHub Actions runs basic unit tests on branch pushes; full integration test suite (TC-01, TC-02, AI-01) runs on Pull Requests targeting the main branch.

- **Regression Testing & Pass/Fail Rules:**
  - Execution Phase: Full test suite runs across all 5 in-scope features each time a branch is merged to main.
  - Pass/Fail Condition: A test case passes only when the actual outcome exactly matches the expected result. Any mismatch is logged as a defect.
  - Continuation Rule: Critical tests (TC-01 happy path, AI-01 buffer overflow) must pass before further pipeline stage testing proceeds.

- **Test Data Strategy:**
  - Manual: 5 predefined trip prompts covering different destinations (Tokyo, Bangkok, Seoul, London, Bali), group sizes (2–8 people), budgets (RM1,000–RM15,000), and languages (English, Malay).
  - Automated: Parameterized PyTest fixtures supply structured prompt inputs and expected Pydantic schema outputs for repeatable pipeline validation.

- **Passing Rate Threshold:**
  - A minimum of 86.2% of all executed test cases must pass. All critical test cases (TC-01, TC-02, AI-01) must achieve 100% pass rate — no exceptions.

4. CI/CD Release Thresholds & Automation Gates

4.1 Integration Thresholds (Merging to Main)

The main branch must be a stable source of truth, enforced through continuous integration checks:

Checks	Requirements	Project Result	Pass/Failed
Automatic Build	Zero Build Errors	0 Errors	**Passed**

Checks	Requirements	Project Result	Pass/Failed
Unit Tests	100% Passing Rate	100%	**Passed**
Code Quality	Zero Linting Errors (ESLint + Flake8)	2 warnings, 0 errors	**Passed**
Test Coverage	Minimum 81.2% (backend PyTest)	83.4%	**Passed**

4.2 Deployment Thresholds (Pushing to Production)

Checks	Requirements	Project Result	Pass/Failed
Regression Test	Minimum 90%	91.3%	**Passed**
AI Output Pass Rate	Minimum 80% of documented prompts	82%	**Passed**
Critical Bugs	Zero P0/P1 Bugs	0	**Passed**
API Performance	Response Time < 800ms (p95)	620ms	**Passed**
Security	API keys not exposed in build output	Clean	**Passed**

5. Test Case Specifications (Drafts)

Test cases include at least: 1 Happy Case (entire flow), 1 Specific/Negative Case, and 2 Non-Functional (Performance/Load) tests.

Test Case ID	Test Type & Mapped Feature	Test Description	Test Steps	Expected Result	Actual Result
TC-01	Happy Case (Entire Flow): Conversational Trip Planning — End-to-End Pipeline	Verify a user can submit a natural language trip prompt, receive a complete structured itinerary with flights, hotels, and expense split via SSE, and confirm the booking via the payment flow.	1. Open https://daddies-trip.vercel.app/ 2. Enter prompt: '5 days in Tokyo for 4 people under RM5000' 3. Submit and observe SSE progress events 4. Verify partial_itinerary event received 5. Verify partial_flights event received 6. Verify complete event	Complete SSE pipeline executes within 90s. All 4 SSE events received in order. Trip object contains valid itinerary days, flight options, per-person cost, and budget summary. Payment confirmation animation shown. Status: Expected Pass	Pipeline completed in 67s. All 4 SSE events received correctly. Trip object validated against Pydantic schema. Itinerary for 5 days generated. 3 flight options returned. Per-person cost calculated at RM1,247. Payment confirmed with animation. Status: Passed

Test Case ID	Test Type & Mapped Feature	Test Description	Test Steps	Expected Result	Actual Result
			with full trip object 7. Proceed to expense split view 8. Enter valid card details and confirm settlement		
TC-02	Specific Case (Negative): Invalid Card Number — Payment Settlement	Verify the system blocks payment and returns a descriptive error when an invalid card number (fewer than 16 digits) is submitted to /api/settle.	1. Complete trip planning to reach payment screen 2. Enter card number: '4111111111' (10 digits) 3. Click Confirm Payment	Payment blocked with HTTP 400 response. UI displays 'Invalid Card Number — please enter a 16-digit card number.' User can retry. No partial payment state created.	Payment blocked. HTTP 400 returned by /api/settle. UI displayed 'Invalid Card Number' message with retry option. No confirmation animation triggered. User was able to re-enter card. Status: Passed
TC-03	NFR (Performance): SSE Pipeline Response Time Under Concurrent Load	Verify the FastAPI backend maintains acceptable SSE pipeline completion time under 50 concurrent user requests.	1. Use Locust load testing tool 2. Simulate 50 concurrent POST /api/plan-trip-stream requests 3. Use identical valid prompts for all users 4. Record p95 pipeline completion time and error rate	p95 pipeline completion time < 90s. Error rate 0%. All SSE connections receive complete event.	p95 completion time: 84s. Error rate: 0%. All 50 SSE connections received complete event. Railway container auto-scaled to handle concurrent load without timeout. Status: Passed
TC-04	NFR (Performance): API Endpoint Response Time — /api/health & /api/settle	Verify individual API endpoints respond within 800ms under 50 concurrent GET/POST requests.	1. Open Postman / k6 load test 2. Send 50 concurrent GET requests to /api/health 3. Send 50 concurrent POST requests to /api/settle with valid payloads	Average response time < 800ms. p95 < 800ms. 0% error rate for both endpoints.	Average response time: /api/health 42ms, /api/settle 312ms. p95: /api/health 68ms, /api/settle 487ms. 0% error rate. Status: Passed

Test Case ID	Test Type & Mapped Feature	Test Description	Test Steps	Expected Result	Actual Result
			4. Record average and p95 response times		

6. AI Output & Boundary Testing (Drafts)

6.1 Prompt/Response Test Pairs

Three prompt/response test pairs with defined acceptance criteria:

Test ID	Prompt Input	Expected Output (Acceptance Criteria)	Actual Output	Status
AI-01	Submit a 9,000-word travel blog post as a trip prompt (6x the 1,500-word limit) to the Orchestrator.	Prompt is truncated to 1,500 words before reaching any LLM agent. Pipeline continues normally with the truncated input. No 500 errors. Planner Agent returns a valid itinerary JSON.	Orchestrator truncated prompt to 1,498 words. Analyzer Agent processed the truncated input, extracted valid fields. Planner returned a complete 5-day itinerary. No errors raised.	**Passed**
AI-02	Submit a valid structured trip prompt: '3 days in Bangkok for 2 people under RM2000' to /api/plan-trip-stream.	Booking Agent returns a trip object where all cost fields are numeric (float), no field contains the repeated hallucination value of RM25, flight routes have distinct departure and arrival airports, and all day numbers are sequential.	Trip object returned with valid numeric costs. Edge Agent detected and corrected 1 instance of RM25 repetition in activity costs. Flight routes had distinct airports. Day numbers were sequential (1, 2, 3).	**Passed (after Edge Agent correction)**
AI-03	Submit a prompt missing the budget field: 'Trip to Seoul for 3 people next month' to /api/plan-trip-stream.	Analyzer Agent detects missing budget field. Pipeline halts and emits clarification SSE event with message specifying the missing field. No Planner Agent call is made. User is prompted to re-submit with budget.	Analyzer correctly identified missing budget. clarification event emitted: 'Please specify your total budget for the trip.' Pipeline halted. No downstream agents called. Frontend displayed clarification message.	**Passed**

6.2 Oversized / Larger Input Test

Maximum Input Size	1,500 words (enforced by Orchestrator before any LLM call)
Input Used While Testing	9,000 words — a 6x oversize travel blog post prompt (AI-01)
Expected Behavior	Truncate prompt to 1,500 words; pipeline continues normally with truncated input
Actual Behavior	Orchestrator truncated to 1,498 words. All 4 agents executed successfully. complete SSE event received with valid trip object.
Status	**Passed**

6.3 Adversarial / Edge Prompt Test

An ambiguous/malformed prompt was submitted to test system resilience:

Adversarial Prompt	Submit: 'asfghjkl trip ??? people ... money' — a malformed, structurally invalid prompt.
Expected Behavior	Analyzer Agent fails field extraction. Emits clarification SSE event listing all 4 missing fields (destination, dates, group size, budget). No downstream agents called.
Actual Behavior	Analyzer identified 0 extractable fields. clarification event emitted with message: 'Please provide: destination, travel dates, number of participants, and total budget.' Frontend showed clarification UI. Pipeline halted cleanly.
GLM Response Handling	Analyzer Agent's LLM call returned a JSON clarification object. System did not raise an exception or emit an error event. User experience remained stable.
Status	**Passed**

6.4 Hallucination Handling

DaddiesTrip addresses LLM hallucination through a dedicated Edge Agent — a deterministic Python heuristic layer that runs after the Booking Agent before the complete SSE event is emitted. Detection and mitigation strategies:

- **RM25 Cost Repetition Bug:** Edge Agent scans all cost fields in the trip object. If more than 2 activity cost fields contain the value 25.0, it is flagged as a hallucination and replaced with a sampled cost from the valid cost distribution of the other activities.
- **Invalid Flight Route Detection:** Edge Agent checks that the departure airport code does not equal the arrival airport code on any flight option. If identical, the arrival airport is corrected based on the trip destination.
- **Missing Day Numbers / Locations:** Edge Agent verifies that each itinerary day object contains a non-null day number (sequential from 1) and a non-empty location string.

Missing fields are flagged and the pipeline emits an error event rather than returning a corrupt trip object.

- **JSON Repair:** base_agent.py applies json_repair (auto-repair library) to all LLM responses before parsing. This handles common JSON hallucinations such as trailing commas, unquoted keys, and truncated output.
- **Prompt Engineering Guardrails:** All agent system prompts explicitly instruct the LLM to return only valid JSON, prohibit fabricating names or prices, and enforce a defined output schema. max_tokens limits are set per agent to prevent truncated JSON output.

© UMHackathon 2026 | umhackathon@um.edu.my